

HELEP

Hybrid Emergency & Localization Emergency Platform

L4 Design Process Document

SEN3244 — Software Architecture
Engr. Tekoh Palma | May 2026

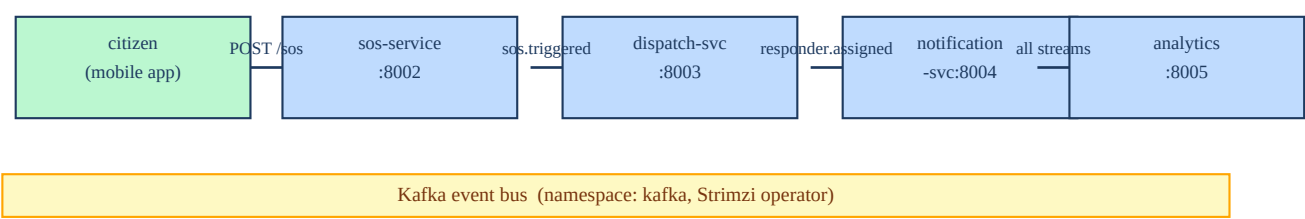
1. Project Specification

HELEP is a mobile-backed emergency response platform built to reduce the time between a citizen raising an alarm and a trained responder arriving on scene. When someone feels unsafe, they open the app and hit SOS. The system picks up their GPS coordinates, finds the nearest free responder in the area, sends dispatch instructions over a Kafka event bus, and pushes a confirmation notification — all without any human dispatcher involved. A companion analytics view lets police administrators see incident density per zone so they can redeploy units proactively.

Four groups of people use the platform. **Citizens** trigger and cancel SOS alerts from a mobile client. **Responders** (police, ambulance, fire units) receive dispatch assignments and confirm they are en route. **Police administrators** access the analytics dashboard to study incident patterns. **Platform operators** watch service health metrics, manage deployments, and run the CI/CD pipeline.

The business case sits on three pillars: faster response times through automated proximity-based dispatch; platform resilience through Kafka-backed event replay so no SOS is ever silently dropped; and operational transparency through a full audit trail of every event that flows through the system.

Figure 1 — SOS saga happy-path event chain



2. Requirements Analysis

2.1 Functional Requirements

Ref	Requirement	Source
F-01	Citizens can register with a phone number, password, and role (citizen / responder / police).	SRS §2.1
F-02	Authenticated citizens can trigger an SOS with GPS coordinates and a danger zone (online or offline).	SRS §2.2
F-03	Citizens can cancel an active SOS. A compensating event is published and the assigned responder is released.	SRS §2.3
F-04	System selects the nearest free responder and reserves them atomically. Multiple incidents can share a responder.	SRS §2.4
F-05	Responders can confirm a dispatch, transitioning the incident to EN_ROUTE.	SRS §2.4
F-06	System detects when a victim's location is inside a registered danger zone and emits an alert.	SRS §2.6
F-07	Notification service delivers alerts on assignment and on zone entry.	SRS §2.5
F-08	In offline mode the system sends an SMS acknowledgement immediately after SOS trigger.	SRS §2.5
F-09	Analytics service exposes event counts, zone summaries, and a crime density map.	SRS §2.7

Ref	Requirement	Source
F-10	Every service exposes /healthz and /readyz probes for Kubernetes health checking.	SRS §2.8.

2.2 Non-functional Requirements

NFR	Attribute	Acceptance criterion
N-01	Availability	99.9% uptime per service. Kafka keeps messages until the consumer comes back up.
N-02	Response time	SOS trigger to notification delivered in under 1 second at 100 concurrent requests.
N-03	Scalability	HPA scales any service from 1 to 3 pods automatically when CPU exceeds 70%.
N-04	Security	JWT HS256 on all write endpoints. Secret never in code or images — injected via K8s Secret.
N-05	Data integrity	A responder can only be assigned to one incident at a time (atomic SQL UPDATE WHERE busy=0).
N-06	API usability	All responses are JSON. Status codes follow RFC 9110 (201, 401, 404, 409, 503).
N-07	Portability	Everything in containers. Config via env-vars only. Runs on any CNCF-certified cluster.

2.3 Constraints and Risks

Constraint (SRS §4)	Risk	How the design addresses it
§4.1 — One responder per incident	Two different pods read the same database row. A responder is busy and another one is not.	Atomic DB updates. Only one responder can be assigned to an incident. WHERE busy=0 AND busy=1.
§4.2 — SOS to notification	A chain of synchronous HTTP calls between services.	Inter-service calls go through Kafka topics. HTTP calls are optional. Each service publishes its own events.

3. Architecturally Significant Requirements

I used a simplified Utility Tree (Bass, Clements, Kazman — *Software Architecture in Practice*) to find the three requirements that most constrain the design. These are the ones where getting it wrong forces a complete rethink of the system.

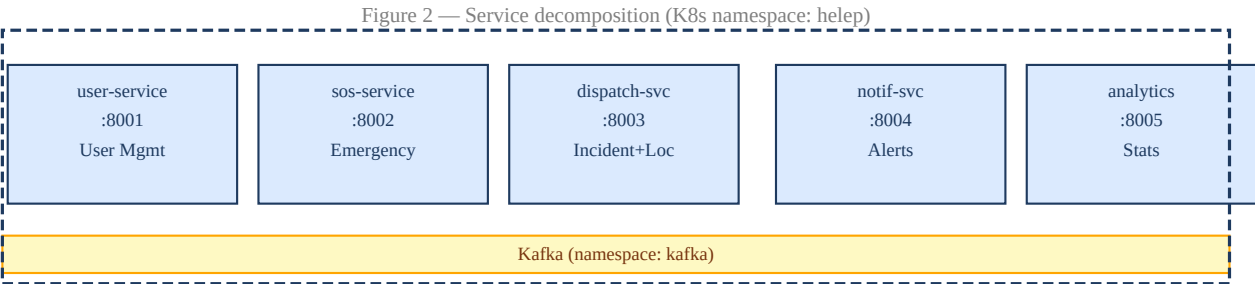
ASR	Quality attribute	Reference	Why it drives the architecture
A-1	Availability / Reliability	N-01, N-02, §4.2	A dropped SOS is a life-safety failure. Any architecture that can silently lose
A-2	Data integrity (no double dispatch)	N-05, §4.1	Two units responding to the same incident leaves another unattended. The
A-3	Independent scalability	N-03	SOS volume spikes during public events or disasters. If every service must

4. Component Identification

4.1 Components named in the SRS

The SRS lists eight logical components: User Management, Emergency Component, Incident Report and Response, Localization, Alert Management, Alert Delivery, Feedback and Review, and Analytics and Statistics. Feedback and Review is explicitly out of scope in this sprint.

4.2 How those components become five services



Service	Port	SRS components inside	Why merged or kept separate
user-service	8001	User Management	Pure identity boundary — signup, login, JWT issue. Kept separate so
sos-service	8002	Emergency Component	Single writer for the SOS lifecycle. The Outbox-lite pattern requires th
dispatch-service	8003	Incident Report & Response + Localization	Location distance and responder selection are one atomic operation
notification-service	8004	Alert Management + Alert Delivery	Pure side-effect consumer — no domain state of its own. Isolated so
analytics-service	8005	Analytics & Statistics	Read-only sink of all Kafka streams (CQRS read model). Isolated so

5. Architectural Style

The brief prescribes Microservices and Event-Driven Architecture. This section defends those choices against two alternatives and maps the trade-offs directly to the ASRs.

Monolith — rejected

One FastAPI process with all five modules inside. The appeal is simplicity: no message broker, no network calls, easy transactions. It passes A-2 trivially because everything shares process memory. It fails A-3 immediately — you cannot scale the SOS write path independently from the analytics read path. It also fails A-1: a bug in the analytics module can crash the process and drop live emergency calls. A monolith is the right choice for early-stage products with a small team and predictable load — neither applies here.

Synchronous microservices (REST fan-out) — rejected

Five services that call each other over HTTP. sos-service calls dispatch-service which calls notification-service before returning to the client. This is clean and easy to trace in logs. The fundamental problem is that latency compounds: if dispatch-service is under load and takes 400 ms, the citizen's SOS endpoint takes at least 400 ms — violating N-02. Worse, if dispatch-service restarts mid-call the event is gone forever, violating A-1. You can add retries and circuit breakers on top, but you end up rebuilding a message queue out of HTTP — it's simpler to just use Kafka.

Microservices + Kafka (chosen)

sos-service publishes to a topic and returns immediately. dispatch-service consumes that topic when it's ready. If dispatch-service is down, the message sits on the partition and is replayed when the pod restarts (auto_offset_reset=earliest). Each service has its own consumer group so a slow analytics consumer cannot block the dispatch consumer. The cost is operational complexity: we need a Kafka cluster, offset management, and idempotent handlers. That cost is justified for a safety-critical platform where a lost event is unacceptable.

Option	A-1 (no lost events)	A-2 (single dispatch)	A-3 (scale)	Verdict
Monolith	Fail — shared crash point	Pass — shared memory	Fail — scales as one	No
REST fan-out	Fail — lost on restart	Partial — needs retry	Pass — separate deploys	No
Kafka (chosen)	Pass — partition replay	Pass — atomic SQL	Pass — per-partition	Yes

6. Patterns Applied

Six patterns were pre-built in the starter. Two more were added in this submission. Full code citations are in patterns.pdf.

#	Pattern	Files	Problem solved
A.1	Choreographed SAGA	sos/main.py, dispatch/main.py, notif/main.py	Coordinates a multi-step transaction across three services. Cancel idempotently.
A.2	Pub/Sub (Kafka)	app/events.py — all services	Decouples services in time. Messages survive consumer restarts via Kafka.
A.3	Repository	app/db.py — all services	Hides SQLite SQL behind named functions. Swap to Postgres by config.

#	Pattern	Files	Problem solved
A.4	Strategy	dispatch-service/app/matching.py	Three responder-selection algorithms (Nearest, Credibility, RoundR
A.5	Outbox-lite	sos-service/app/main.py trigger()	DB write and Kafka publish in same async block to minimise dual-w
A.6	Circuit Breaker	app/events.py CircuitBreaker	Fast-fails Kafka publish calls when broker is unreachable. Prevents
B.1	CQRS	analytics-service/ (whole service)	Read model separated from write path. Analytics queries don't cont
B.2	Bulkhead	events.py consume() group_id per service	independent consumer groups so a lagging analytics consumer can

Figure 3 — Compensating transaction (cancel saga)



7. Architecture Decision Records

ADR-01: Partition key = incident_id, partition count = 3

Context

Every KafkaTopic needs a partition count and each message needs a key. Options were: no key (random distribution), key by user_id, key by incident_id.

Decision

Three partitions per topic, all messages keyed by incident_id (e.g. await publish("sos.triggered", payload, key=iid) in sos-service/app/main.py).

Consequences

Positive — all events for one SOS land on the same partition, consumed in order by the same pod. This is what makes the no-double-dispatch invariant hold across multiple dispatch pods without a distributed lock. Up to 3 pods can process different incidents in parallel. Negative — once you go above 3 replicas per service the extra pods sit idle until you raise the partition count.

Alternatives considered

No key (rejected: breaks ordering, two events for the same incident could be processed by different pods simultaneously). Key by user_id (rejected: a very active user during a crisis could overload one partition while others are idle).

ADR-02: One SQLite database per service on a dedicated PVC

Context

Each service needs persistent storage. Options: one shared PostgreSQL cluster vs. an embedded SQLite file per service on its own PersistentVolumeClaim.

Decision

SQLite per service, each mounted at /data.db on a ReadWriteOnce PVC with storageClass: standard and a helm.sh/resource-policy: keep annotation.

Consequences

Positive — services are completely isolated at the data layer; a schema change for one cannot break another; a database crash is scoped to one service. Negative — SQLite only supports one concurrent writer, which means a PVC with ReadWriteOnce limits the service to one writer pod at a time. Fine for this sprint; not acceptable for production traffic. Replace SQLite with managed PostgreSQL in a real deployment.

Alternatives considered

Shared PostgreSQL (rejected: a single slow query or migration could affect all five services simultaneously — breaks the isolation microservices are supposed to provide). MongoDB (rejected: no

benefit over SQLite for small tabular datasets; adds operational overhead for no gain).

ADR-03: Helm umbrella chart with one sub-chart per service

Context

Options for packaging a 5-service K8s deployment: raw YAML manifests, one Helm chart per service with separate releases, or an umbrella chart with sub-charts.

Decision

One umbrella chart at charts/helep/ with five sub-charts. The umbrella owns the shared ConfigMap, Secret, and Ingress. Each sub-chart owns its Deployment, Service, HPA, and PVC. A single helm upgrade deploys everything atomically.

Consequences

Positive — one command upgrades all five services together; --atomic rolls everything back if any resource fails; environment-specific overrides live in one values file. Negative — a template error in one sub-chart blocks the whole release; requires helm dependency build before first install.

Alternatives considered

Raw manifests (rejected: no templating, environment promotion is manual copy-paste). Separate releases (rejected: no atomic cross-service rollback; shared resources like the JWT Secret would have to be managed outside Helm).

8. Trade-offs and Improvements

Weakness	Why it exists	What I would do with more time
SQLite doesn't safely handle multiple writers.	SQLite is a file-based engine. Replicating writes to multiple instances is a problem.	Replace SQLite with a more robust database like PostgreSQL or Redis.
The Outbox-lite has a crash window.	SQLite has a process-wide lock. If the process crashes, the lock is held by the OS.	Use a distributed transactional outbox like Kafka or a dedicated outbox service.
CircuitBreaker state is per-pod.	StatefulSet can have different replicas with different states.	Use a shared Redis key for the circuit breaker state.

9. Checklist

- Section 1: project spec with 4 user classes and business value
- Section 2: FR table (10 rows), NFR table (7 rows), constraints table (2 rows)
- Section 3: 3 ASRs with attribute, reference, and justification
- Section 4: SRS components listed, 5-service decomposition with rationale
- Section 5: style defended against monolith and REST SOA with trade-off table
- Section 6: 8 patterns (6 starter + CQRS + Bulkhead), file references
- Section 7: 3 ADRs (partition key, SQLite, umbrella chart)
- Section 8: 3 weaknesses with concrete improvement proposals

- 3 architecture diagrams (Figures 1, 2, 3)
- All decisions trace to SRS line, NFR, or ASR
- patterns.pdf delivered alongside this document